B+ Tree Implementation and Performance Comparison

1. Introduction

In this assignment, I implemented and compared three ways of storing a B+ Tree: a memory-only version, a disk-only version, and a version that uses both memory and disk. I ran the program on the given CSV dataset containing 10 million records. Each record has a unique alphanumeric key and a payload. I compared the versions using insertion time, average search time, tree height, storage size, and disk page I/O where applicable.

2. Dataset and assumptions

Item	Details
Number of records	10,000,000
CSV columns	key and payload
Node/page size reported by program	5,400 bytes
Maximum keys per node	31
Tree height reported	6 for all three versions

I used the values printed by my program for the node/page size and maximum keys per node. The results below are from the full run shown in the terminal output. Storage size is the value reported by the program, so it should not be treated as a complete measurement of total RAM used by the process.

3. Results

Metric	Memory-only	Disk-only	Memory + Disk
Insertion time	33.597 s	573.815 s	491.811 s
Average search time	3.465 µs/key	30.626 µs/key	24.454 µs/key
Tree pages/nodes	4,820,269	482,069	482,069
Storage size	2,618,598,808 bytes (2,497.290 MiB)	2,603,172,600 bytes (2,482.579 MiB)	2,603,172,600 bytes (2,482.579 MiB)
Tree height	6	6	6
Disk page reads	0	50,845,201	27,451,095
Disk page writes	0	10,964,096	10,931,391
Cache hits	0	157	23,394,263

4. What I observed

Insertion time: The memory-only version took the least time, at 33.597 seconds. The disk-only version took 573.815 seconds, while the hybrid version took 491.811 seconds. This shows that writing and reading pages from disk added a noticeable cost in this run.

Search time: The average search time was 3.465 µs/key for memory-only, 30.626 µs/key for disk-only, and 24.454 µs/key for the hybrid version. The hybrid version was faster than disk-only in the measurements I got, although it was still slower than keeping the tree in memory.

Disk I/O and cache: The memory-only version reported no disk page reads or writes. The disk-only version reported 50,845,201 page reads. The hybrid version reported 27,451,095 reads and 23,394,263 cache hits. This suggests that keeping pages in memory helped avoid some disk reads.

Tree height: All three versions reported a height of 6. Since the reported height was the same, the timing differences appear to be mainly related to how the data was stored and accessed.

Storage size: The two disk-backed versions reported 2,603,172,600 bytes, while the memory-only version reported 2,618,598,808 bytes. These are the storage figures printed by the program and may not include all memory-management overhead.

5. Conclusion

From my results, the memory-only B+ Tree was the fastest option for both insertion and search. Its main limitation is that the tree needs to remain in memory. The disk-only version was the slowest in this experiment, which is consistent with the large number of page reads reported by the program. The hybrid version performed better than disk-only and recorded many cache hits, so using memory to keep frequently accessed pages can improve performance. Overall, the best option depends on whether speed, available RAM, or disk-based storage is the main requirement.

6. Notes on the experiment

The numbers in this report were taken from the terminal output after running the program on 10 million records. The results can change with different hardware, compiler settings, or disk-cache conditions. For a fair comparison, all three versions should be run on the same computer with the same dataset and compiler settings. I would submit the source code and the generated benchmark_results.csv along with this report.